

Sistemas Operativos

2026

Práctico 1

Comandos, redirección, pipes, señales y llamadas al sistema

Lecturas requeridas:

1. Capítulos 1, 2 y 3 de las Notas del curso
2. Capítulo 1 del libro **Modern Operating Systems**. A. Tannenbaum, H. Bos.

Ejercicios:

1. Realizar los siguiente comandos en el shell (de comandos) de su sistema (familia UNIX).
 - a. Listar los procesos que están en estado de ejecución (ver el comando ps, hacer man ps para leer el *manual* del comando). Determinar el *process id (pid)* del primer proceso lanzado (*init* en sistemas tipo UNIX) por el kernel del SO.
 - b. Listar los usuarios conectados (logged) al sistema con el comando who y ver la salida del comando who am i.
 - c. Determinar cuál es el *directorio (carpeta) corriente*. Usar al menos dos comandos diferentes.
 - d. Determinar cuál es su directorio de trabajo de usuario (*home*) (ver comandos cd).
 - e. Listar los metadatos de los archivos (con sus detalles) de la carpeta corriente y de la carpeta raíz sin cambiarse de directorio.
 - f. Listar el contenido de algún archivo (sin usar un editor). También hacerlo por páginas (o pantallas).
 - g. Listar y analizar las *variables de ambiente* de su sesión (ver comando env).
 - h. Determinar en qué directorios el shell busca los comandos (ver PATH variable).
 - i. EL comando `which <cmd>` muestra el directorio (*path*) donde se encuentra el comando.
 - j. Crear un archivo con al menos tres comandos diferentes.
 - k. Crear un *enlace simbólico* o *alias* a un archivo o carpeta (ver el comando ln).
 - l. Eliminar el *enlace simbólico* creado. ¿Afecta al archivo o carpeta original?
- Determinar la diferencia entre *symbolic links* y *hard links*.
 - m. Crear un programa (hello) en C que muestre los argumentos recibidos en la línea de comandos y ejecutarlo.
¿Porqué debe ejecutarlo con `./hello arg1 ... argn`?
2. Comandos de shell: Redirección, secuencia y concurrencia.
 - a. Crear dos archivos de texto con una lista de direcciones de correo electrónico (una por línea) que puedan tener duplicados. Generar un archivo con la lista ordenada y sin duplicados. Ayuda: ver sort y uniq. Los archivos temporales deberían crearse en /tmp.
 - b. Idem al anterior pero hacerlo en un único comando usando el operador secuencial.
 - c. Escribir un comando que filtre y cuente las direcciones terminadas en *google.com*. Ver grep y wc.
3. Repetir lo del primer ítem del ejercicio anterior pero sin usar archivos temporales (ver pipes).
4. Escribir un **shell script** con los comandos del último ítem del ejercicio anterior. Probar su ejecución mediante el comando sh y luego agregando la línea de intérprete. Dar permisos de ejecución (con el comando chmod +x).
5. Escribir un programa C que retorne como *exit status* el valor ingresado en la línea de comandos. Ejemplo de invocación: `./myprog 1`.

- a. Escribir un comando para ejecutar el programa y determinar su valor de salida. Ayuda: La variable `$?` toma el valor de salida del último comando ejecutado.
 - b. Escribir un comando que ejecute otro si `myprog` finaliza con éxito (0).
 - c. Escribir un comando que ejecute otro si `myprog` no finaliza con éxito.
6. La biblioteca estándar de C ofrece la función `int system(const char* string)`, la cual crea un subproceso (*child*) generado por el comando pasado en su argumento. Definir una función con su propia versión de `system()` en términos de las llamadas al sistema `fork()`, `exec()` y `wait()` (ver páginas del manual).
7. Implementar un programa en C que lance un proceso hijo y luego el proceso padre envíe un string al proceso hijo por medio de un **pipe** (ver ``pipe()`` `syscall`). El proceso hijo deberá mostrar por la salida estándar el string recibido.
8. Idem al ejercicio anterior pero haciendo que los programas que se comuniquen por medio de un FIFO (named pipe). Ayuda: ver el comando `mkfifo`.
9. Escribir un programas `minish.c` que esté en un ciclo de lectura de un comando y lo ejecute hasta que reciba la señal `Ctrl-D` (fin del input).
Debe reconocer y ejecutar los siguientes comandos:
 - a. `cmd1 & cmd2` # ejecución concurrente (de fondo)
 - b. `cmd1 < input_file` # redirección de la entrada estándar
 - c. `cmd > output_file` # redirección de la salida estándar
 - d. `cmd >> output_file` # append en archivo
 - e. `cmd1 ; cmd2` # ejecución secuencial
 - f. `cmd1 || cmd2` # ejecución condicional
 - g. `cmd1 && cmd2` # idem
 - h. `cmd1 | cmd2` # pipe
10. Hacer un programa C que capture la señal `SIGINT`. El manejador de la señal deberá preguntar al usuario si realmente desea finalizar el proceso. En caso de respuesta afirmativa finalizar con un estado de salida 0. En otro caso finalizar con estado -1. Probar enviando la señal al proceso usando el comando `kill`.
11. Hacer un programa C que dado un arreglo de valores enteros, de dimensión N , cree p procesos hijos, los cuales deberán calcular el mínimo valor de la porción que le corresponda (N/p elementos) y enviarlo al padre a través de un pipe. El padre deberá esperar los resultados y computar el mínimo global.
12. Escribir un programa que lea un dato de la entrada estándar. Si no recibe datos luego de N segundos, debería finalizar. Nota: Ver la función de biblioteca `alarm`.